

Klas / naam:

Module ONT-I

Ontwerpen

Over dit document

Blok: A
Opleiding: Team ICO / Software Developer / Breda
Vak(ken): PRO
Weken 5, 7, 11 – 12, 14 – 15

Versiebeheer

Versie	Datum	Auteur	Aanpassingen
0.2	08-10-2020	EB89	Wireframes
0.3	09-11-2020	BR10	H2
1.0	14-11-2020	BR10	H3, 4, 5
2.0	01-03-2021	BR10	Gereed voor nieuwe uitvoer
2.1	10-03-2021	BR10	Alle lessen versie 2 helemaal afgerond
2.2	18-03-2021	BR10	Toevoeging H6 over schoolmail

 **Theorieblok**

 **Opdracht**

 Een term: die je misschien nog niet kent

Inhoudsopgave

I. Over deze module	3
II. Studiewijzer	3
1. Wireframes introductie	4
2. Wireframes bij WEB	7
3. Flowcharts	14
4. Diversen en feedbackmoment	24
5. Wireframes uitbreiden	26
6. Flowcharts revisited	29
III. Feedback-momenten	31
IV. Voor de docent	32
V. Kwaliteit en verbeteracties	32

I. Over deze module

Doelstelling en relevantie

Deze module gaat over de tweede fase van een project/praktijkopdracht; het ontwerpen. Nadat je hebt uitgezocht wat de opdracht precies is (de plan-fase), ga je nog een ontwerp maken. Voor dat ontwerpen zijn verschillende technieken; gaat het om een website, dan maak je meestal een wireframe. Voor WIN-opdrachten wordt vaker met een flowchart gewerkt.

Samenhang met andere onderdelen

Samenhang met de praktijkopdrachten A2 t/m A5.

Materiaal- en middelenlijst

- Pen en papier

Feedback-momenten

Week 12: in het feedback-moment zul je gevraagd worden om uit een aantal voorbeelden het beste wireframe te kiezen, op basis van de theorie. Ook krijg je meerkeuzevragen over de theorie. Voor flowcharts zul je voornamelijk invulvragen krijgen.

Samenhang met de ICO-V



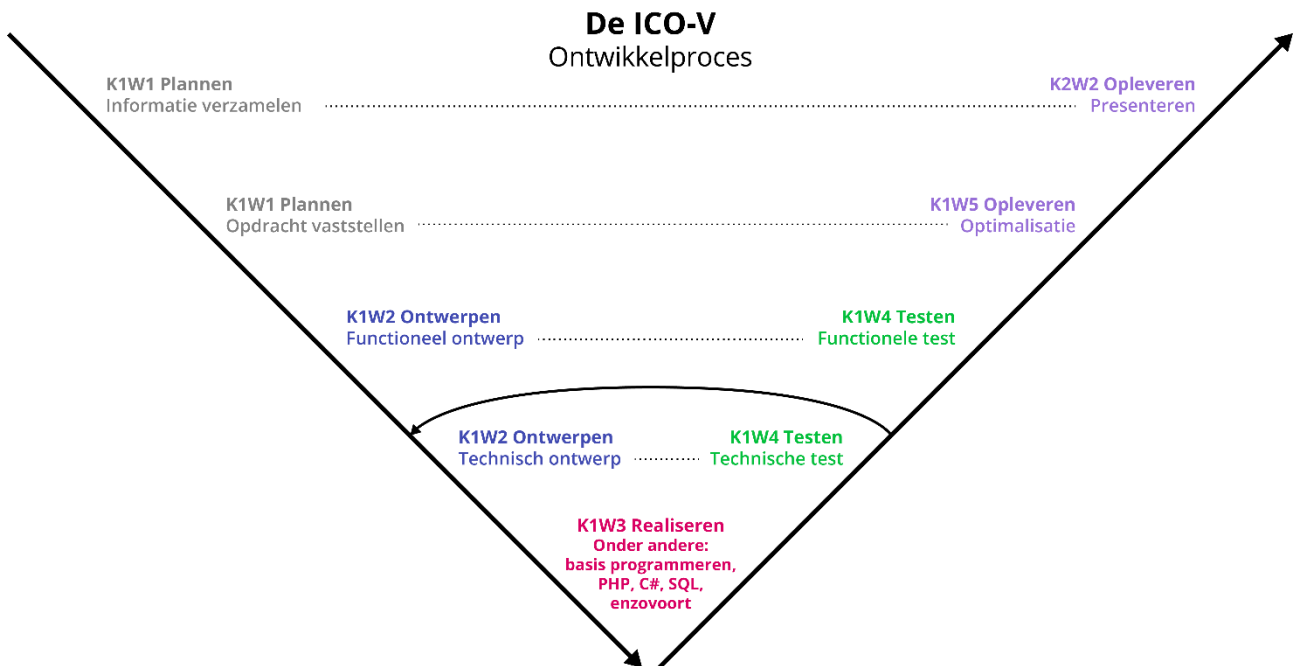
II. Studiewijzer

Week	Hoofdstuk / onderwerpen	Inleveropdracht	Bronnen / websites / literatuur
5	H1 – Wireframes intro	-	
7	H2 – Wireframes bij WEB	Dropplace	
11	H3 – Flowcharts	-	https://en.wikipedia.org/wiki/Flowchart#Building_blocks
12	H4 – <u>Feedbackmoment</u>	-	
14	H5 – Wireframes uitbreiden	-	
15	H6 – Flowcharts revisited	-	

1. Wireframes introductie

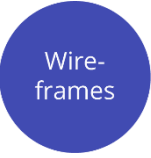
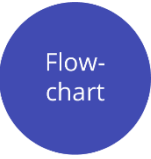
1. Over deze module

In de module WED-I heb je geleerd hoe het ontwikkelproces eruit ziet, dus: welke dingen komen er kijken bij het maken van een stuk software, naast het programmeren.



In deze module, ONT-I, gaan we kijken naar de tweede stap van het proces: “ontwerpen”. Het gaat dan niet over ontwerpen in de zin van “iets kunstigs maken” maar over het zogenaamde “functioneel en technisch ontwerpen”. Je bedenkt hoe de functies van een app eruit moeten zien, en hoe ze gaan werken.

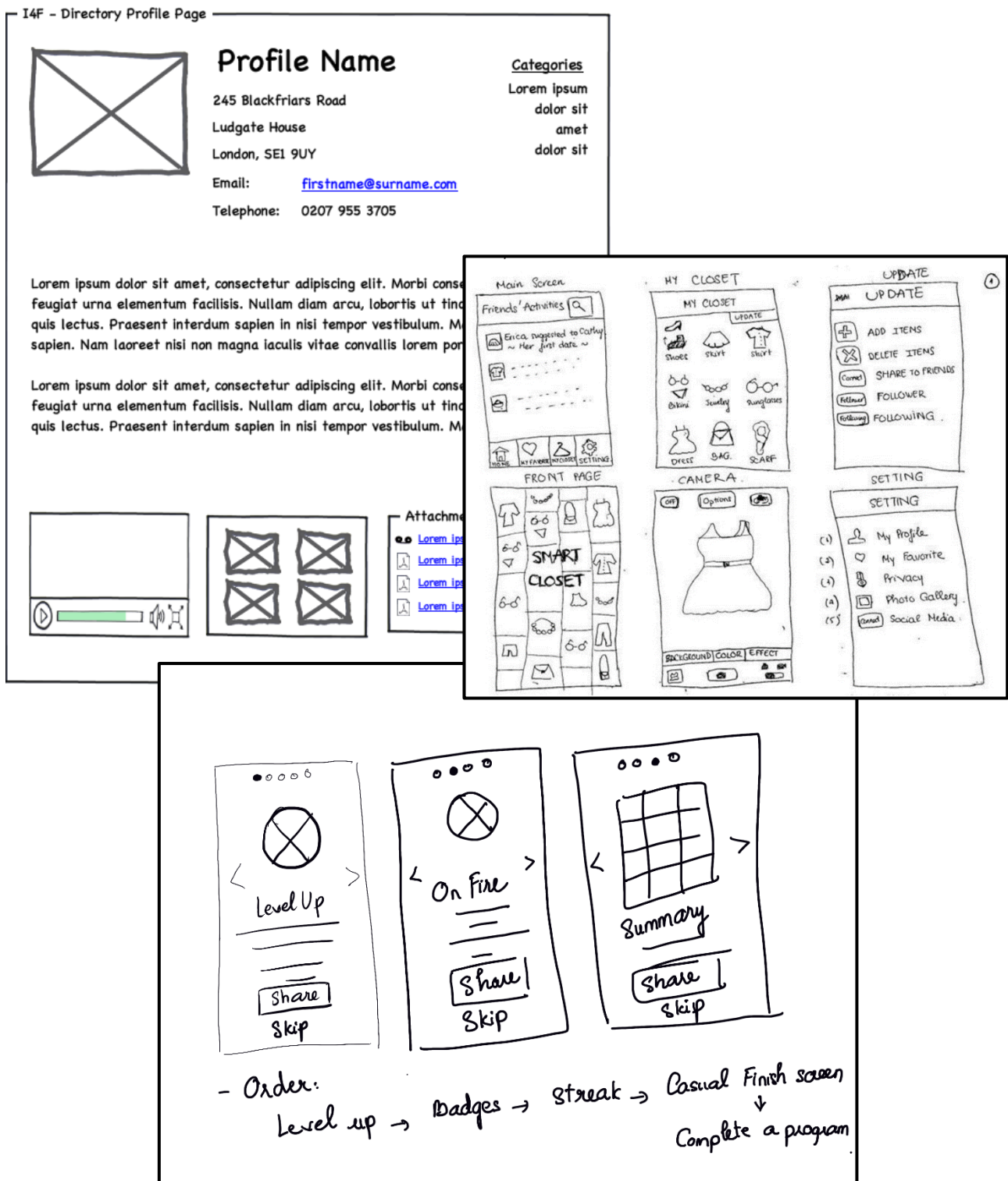
Bij ONT-I gaan we twee activiteiten bekijken:

	Een wireframe is een tekening waarin je vastlegt hoe je app of website eruit komt te zien. Je legt de “lay-out” vast: welke onderdelen komen waar op het scherm. Dit gebruik je vaak bij een WEB-opdracht.
	Met een flowchart teken je uit hoe een applicatie gaat werken qua techniek, welke stappen komen voor in de code en op welke volgorde komen die stappen. Een flowchart past vaak goed bij een WIN-opdracht.

In de volgende praktijkopdracht (A2) ga je een website ontwerpen en maken, daarom kijken we alvast naar het onderwerp “wireframes”.

2. Wat is een wireframe?

Een wireframe is een tekening van de lay-out van een website of applicatie. Lay-out betekent: wat staat waar. Denk aan de positie van het menu, waar afbeeldingen en tekst komen te staan, positie van knoppen en invul-velden, enzovoort.



Een wireframe gaat dus *niet* over dingen als kleurgebruik, lettertypes of andere "styling". Je ziet echt alleen de lay-out terug, dus wat staat waar. Ieder scherm of iedere pagina wordt één wireframe.

3. Regels voor een wireframe

Er zijn een paar regels bij het maken van een wireframe. Herken je deze in de voorbeelden?

- Werk het liefst op papier, echt tekenen dus. Digitale tools zitten het effect van snel schetsen in de weg. Je hoeft niet netjes te tekenen, het gaat erom dat de lay-out zichtbaar is.
- De belangrijkste knoppen, links en invulvelden moeten zichtbaar zijn.
- Schrijf alleen de belangrijkste woorden. Grote stukken tekst kun je weergeven met lijntjes of “lorem ipsum” onzintekst.
- Een afbeelding wordt vaak getekend als een vakje met een kruis erin.
- Geef je pagina's een naam (“home”, “contactpagina”). Schrijf extra uitleg als dat nodig is.

4. Opdracht: natekenen

Kies een app of website die je veel gebruikt en teken minstens twee schermen na als wireframe. Houd de bovenstaande regels in de gaten.

Besteed maximaal 15 minuten aan het natekenen, je moet immers niet te veel details tekenen. Het gaat om een snelle schets van de lay-out.

Als de docent vraag om je wireframes in te leveren, maak dan een *duidelijke* foto. Controleer of de tekening goed zichtbaar is en maak eventueel een nieuwe foto.

5. Opdracht: website ontwerpen

Je krijgt de opdracht om een website te ontwerpen voor een klein bedrijfje. Teken wireframes voor de volgende pagina's:

- Homepage – bovenaan een menubalk, dan een grote foto en daaronder twee blokken tekst naast elkaar met als koppen “Over ons” en “Het bedrijf”. Daaronder komen twee afbeeldingen naast elkaar, en daar weer onder een footer met wat algemene info.
- Medewerkers – zelfde menubalk en foto, daaronder een kleine ronde foto van de directeur, rechts daarnaast persoonlijke info. Hieronder over de hele breedte een stukje tekst over deze persoon. Dezelfde lay-out voor een tweede medewerker. Daaronder dezelfde footer als de homepage.

6. Opdracht: bespreken wireframes

Docent en klas bespreken de gemaakte wireframes. Enkele mogelijkheden:

- Eerst in twee- of drietallen vergelijken met elkaar, de regels en de voorbeelden.
- Docent kiest enkele voorbeelden uit om klassikaal te bespreken en vergelijken met de regels.

Neem in ieder geval deze vragen met elkaar door:

- Voldoen de wireframes aan de opdracht, ben je niks vergeten?
- Zou een buitenstaander begrijpen wat er te zien is?
- Voldoen de wireframes aan de regels van paragraaf 3?
- Zou een klasgenoot hier genoeg aan hebben om de website te maken?

2. Wireframes bij WEB

1. Verder met wireframes

In hoofdstuk 1 van deze module heb je geleerd wat een wireframe precies is. Deze week gaan we verder met het onderwerp, zodat je bij praktijkopdracht A2 zelfstandig goede wireframes kunt tekenen.

Als software developer moet je meer kunnen dan alleen maar coderen. Hier hoort ook ontwerpen bij. Er zijn veel facetten die komen kijken bij het ontwerpen van een (web) applicatie. We noemen dat ook wel het ontwerpen van de “User Experience”, de ervaring die de gebruiker heeft met jouw app. Deze term wordt meestal afgekort tot UX.

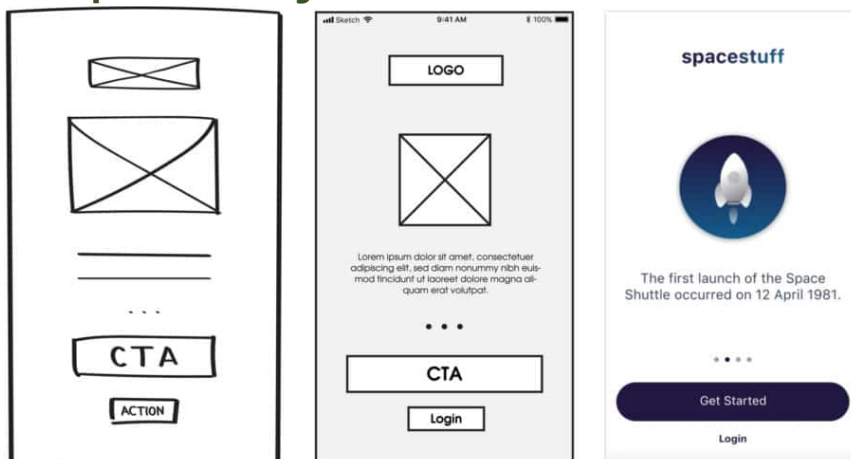
Het gaat dus om meer dan alleen hoe iets eruitziet of hoe het werkt. Het is een totaal plaatje van het gebruik van de applicatie. Er komt onderzoek, ontwerp, ontwikkelen en onderhouden bij kijken. Voorbeelden van klussen die je krijgt als UX-designer zijn:

- Marktonderzoek doen: hoe kan je in een bestaande markt uitblinken. Je bekijkt eventuele vergelijkbare websites en bedenkt waarom sommige websites wel werken en waarom sommige niet
- Je bedenkt hoe iemand zich door je website gaat navigeren (met behulp van bijvoorbeeld flowcharts)
- Je maakt een wireframe van hoe de website er ongeveer uit gaat zien
- Je maakt de website en bedenkt wat fijne manieren zijn om door de website te navigeren
- Je houdt bij hoe je website bezocht wordt en kan op basis daarvan je design aanpassen

Als je wireframes moet tekenen, worden ze beoordeeld op de onderstaande punten. Die gaan we ook behandelen in de komende paragrafen:

12.02	Voor het functioneel ontwerp teken je <u>wireframes</u> waarin de lay-out van de applicatie te zien is, alsook de ‘flow’ tussen de schermen.
Wireframes	De wireframes zijn ‘lo-fi’ getekend; vergelijkbaar met een potloodtekening. Goed te begrijpen, waar nodig voorzien van commentaar. Alle nodige schermen zijn uitgetekend.
Lay-out	Bevatten informatie over de besturing van de applicatie (knoppen en invoervelden). Dit is logisch en compleet.
Flow	De verbinding tussen verschillende schermen is duidelijk, de stappen die gebruiker neemt door de applicatie.

2. Een potloodtekening



Je kunt verschillende soorten wireframes tekenen. Van links naar rechts zie je hierboven:

- **Low-fidelity:** dit is wat we van jullie vragen, een schets die ongeveer aangeeft wat de lay-out wordt. Dus: welke elementen komen waar te staan op de pagina. Qua tekst zie je alleen de allerbelangrijkste woorden terug. Omdat het een snelle schets is, kun je ook makkelijk nieuwe versies maken en dingen aanpassen, als de klant dat nog wil.

- **Medium-fidelity:** als je dat handig vindt, kun je een wireframe ook nog digitaal uitwerken. Hij wordt dan iets netter, en je kunt iets meer details toevoegen. Meestal neem je die moeite pas wanneer je vrij zeker weet dat je ontwerp goed is.
- **High-fidelity:** dit is eigenlijk een kant-en-klaar ontwerp waarin ook kleuren en lettertypes zijn verwerkt. Als software developer maak je meestal niet zo'n gedetailleerd ontwerp, dat is meer het werk van een gespecialiseerde *designer*.

3. Flow

Een applicatie bestaat eigenlijk altijd uit meerdere schermen of pagina's. Je tekent dus ook niet één wireframe, maar er komt voor ieder scherm een tekening. Je kunt dan ook mooi aangeven hoe de "flow" in je applicatie gaat, ofwel: hoe navigeert een gebruiker van de ene pagina naar de andere.

In het voorbeeld hieronder zie je dat met pijlen is aangegeven wat er gebeurt wanneer je op een link of knop drukt. De pijl wijst naar de pagina waar je dan naartoe gaat:



Je kunt dus concluderen dat het handig is om meerdere schermen op een pagina te tekenen. Een bijkomend voordeel is dat je dan wat kleiner moet tekenen, en dus niet te veel details kwijt kunt in je tekening.

4. Lay-out en details

Een wireframe toont de lay-out van je pagina's: wat komt waar te staan. Je moet hier niet te *veel* details in verwerken, want een wireframe gaat niet over de precieze teksten of plaatjes. Het gaat zeker niet over een rand die 2px of 3px dik moet worden.

Maar andere details zijn wél belangrijk. In het voorbeeld uit de vorige paragraaf kun je heel goed zien hoe de applicatie “bestuurd” zal worden. Je ziet de knoppen en invulvelden voor de gebruiker.

We kunnen dus concluderen:

- ✓ Wel laten zien waar de gebruiker op kan klikken en wat hij kan invullen.
- ✗ Géén details zoals lange teksten, kleuren, op de pixel nauwkeurig, precies recht, enzovoort.

5. Opdracht: zelf aanvullen

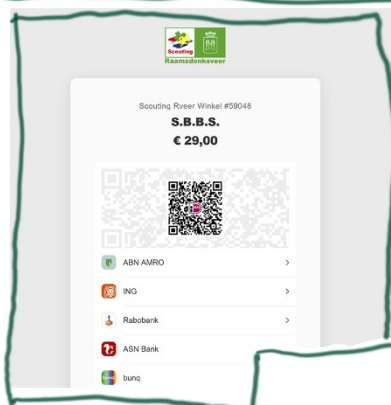
In deze opdracht gaan we kijken naar de wireframes van een eenvoudige webwinkel. Op de volgende bladzijde zie je wireframes van het betaalproces. Op scherm nummer 4 moet een formulier komen, maar hier staat alleen nog een groot leeg vak. Het is dus onvoldoende duidelijk hoe de applicatie “bestuurd” zal worden. Je ziet ook de échte webpagina. Teken het formulier-vak correct in.

(HEADER, ZIE EERDER)

ORDER	~	~	€00,00
TOTAAL			€00,00

FORM

4. ORDER



5. EXTERN BETAALSCHEM

6. BEVESTIGING

(HEADER)

ORDER#1234			
NAAM (TEAM)			
~	~		€ 00,00
TOTAAL			€00,00

Het echte scherm nummer 4:

Winkel van Scouting Raamsdonksveer



Bestellen

Vest	Unisex XS	€29.00
Totaal		€29.00

Naam

E-mailadres

Team

Overige

[Bestellen & betalen >](#)

6. Opdracht: onderzoek

Veel websites hebben ongeveer dezelfde lay-out. Je zult bijvoorbeeld weinig websites tegenkomen die hun menubalk rechtsonder hebben verstopt.

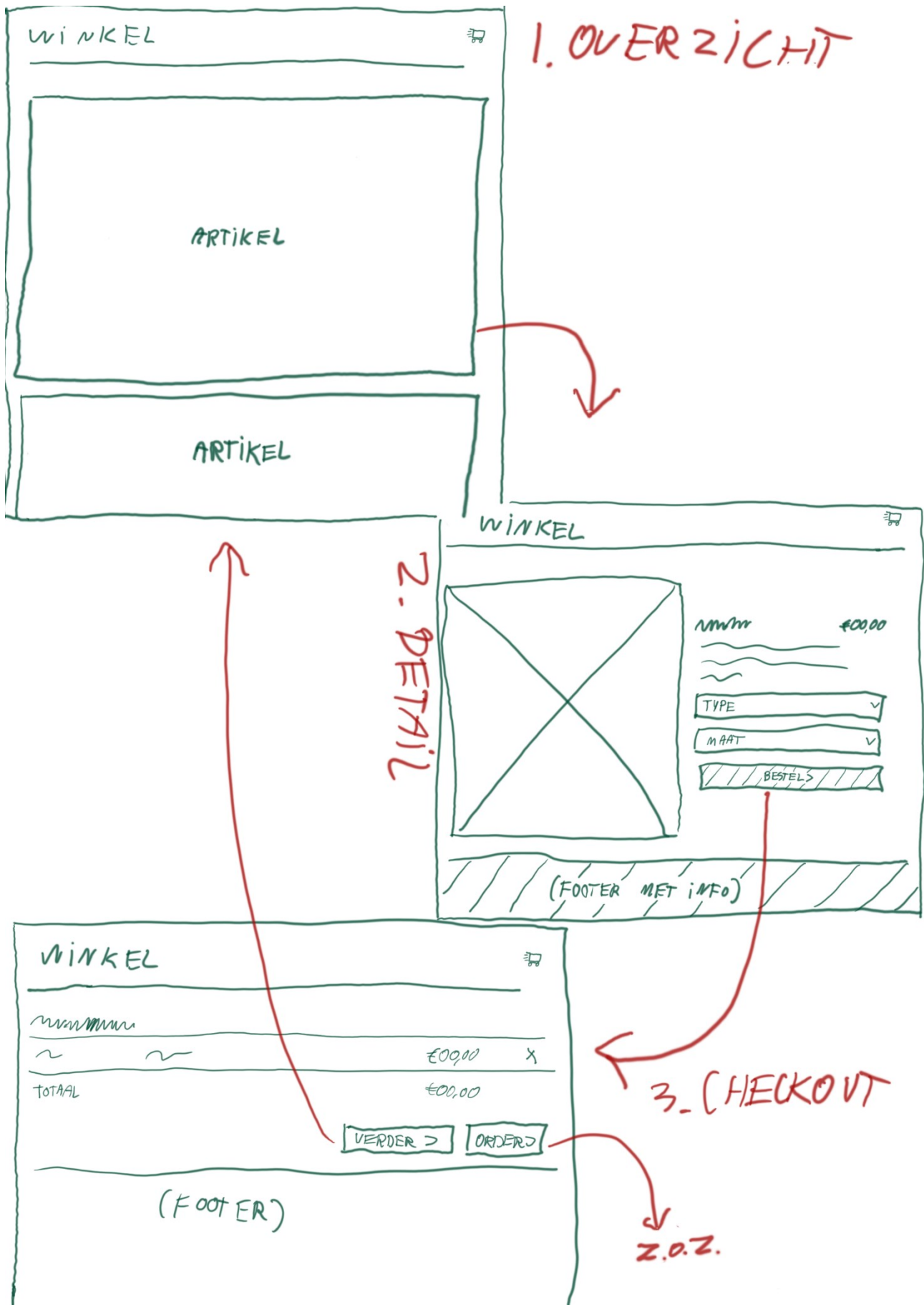
1. Bekijk een aantal websites, en schrijf op wat je opvalt. Welke dingen zijn vaak hetzelfde? Kijk naar de aanwezigheid van onderdelen, of de positie op de pagina die vaak hetzelfde is. Je mag zelf een aantal websites uitkiezen, of bezoek er een paar uit dit lijstje:
 - <https://curio.nl/>
 - <https://nu.nl/>
 - <https://telegraaf.nl>
 - <https://ikea.nl/>
 - <https://tweakers.net/>
 - <https://bol.com/>
 - <https://knvb.nl/>
2. Docent en klas bespreken de uitkomsten. Als klas trekken jullie een conclusie over welke “regels” er zijn, of dingen die bijna altijd hetzelfde werken in de lay-out van een website.

7. Opdracht: invullen wireframe

We gaan terug naar de kleine webwinkel van opdracht 5. Op de volgende pagina vind je drie wireframes:

1. De overzichtspagina, hierop zie je alle artikelen onder elkaar. Ieder vak is één artikel.
2. Detailpagina, een klik op een artikel brengt je naar de detailpagina.
3. Winkelwagentjes.

Scherf 1 bevat nog te weinig details. Ga op onderzoek bij andere webwinkels; hoe ziet zo'n overzichtspagina er meestal uit? Teken dan een nieuwe versie van de overzichtspagina, of vul het bestaande wireframe aan.



Inleveropdracht: Dropplace

Dropplace is een winkelketen van tien winkels die in verschillende Brabantse steden drop verkoopt. Je kunt het zo gek niet bedenken of ze hebben het; eurodrop, trekdrops, autodrops, fietsdrops, driewielerdrops, vingerdrops en klompendrops, enzovoort!

Nu heeft de directrice Susie Soetekouw het volgende idee: omdat ze haar bedrijf wil laten groeien gaat ze ook online haar drop verkopen, hiervoor wil zij een dropsite gaan starten. Een dropsite waar dropliefhebbers uit de hele wereld hun drop kunnen bestellen. Als belangrijkste doelgroep ziet zij Nederlanders die in het buitenland wonen en werken. Onder het motto "Schep je eigen drop" wil ze een webshop starten.

Maak een aantal low-fidelity wireframes van de website Dropplace. Het gaat om een desktop-website. Hou hier rekening met de verschillende pagina's en knoppen die de website zal hebben. Je tekent minimaal deze pagina's uit:

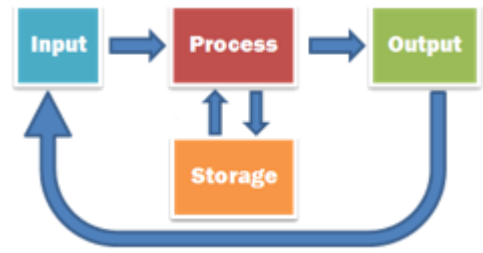
- Home – met een grote foto die de aandacht trekt, en daaronder drie 'uitgelichte' producten, daar weer onder komt wat algemene tekst over het bedrijf.
- Producten – links een balk met filters (smaken, soorten, fabrikant, prijs, etc.) en midden en rechts zie je dan alle producten met een titel, foto en prijs.
- Contact – een contactformulier en ook informatie als telefoonnummer, retouradres, enzovoort.

3. Flowcharts

1. Flowcharts: wat en waarom

In module PRB-I heb je geleerd over het principe van “input-process-output”:

- Een programma start altijd door een bepaalde input: je drukt op een knop, de lichtsensoren meet iets, je voert tekst in bij een console-programma of er wordt een bestand ingelezen.
- Dan vindt een proces plaats waarin het programma iets *doet* met die input; een bepaalde berekening, toetsen van voorwaarden, enzovoort. Dit is de kern van je programma.
- Tenslotte geeft je programma output: er gaat een lichtje branden, een geluid wordt afgespeeld, je ziet tekst op het scherm of er is een bestand weggeschreven.



In het begin maakte je veel programma's waarin de stap van “process” vrij eenvoudig was. Denk bijvoorbeeld aan “Is er voldoende brandstof? Print dat we heen-en-weer kunnen vliegen.”

Maar langzaam wordt het proces dat je moet programmeren steeds complexer. Je kunt het dan niet meer altijd ‘voor je zien’. Wanneer een programma te complex wordt om in één keer te overdenken, kun je een flowchart gebruiken om het proces uit te tekenen. De definitie van een flowchart is dan ook;

“Een schematische voorstelling van een proces”, of “een diagram dat een proces visualiseert”.

Flowcharts bij complexiteit

Het eerste doel van een flowchart is dus voor jezelf: om je eigen gedachten op papier te zetten en het proces te visualiseren. Zo hoef je niet alles in je hoofd te doen, en raak je minder snel ‘de weg kwijt’ tijdens het programmeren. De flowchart is een hulpmiddel om overzicht te houden, om complexiteit te vangen op papier. Je zult merken dat, wanneer je meer ervaring krijgt, je minder snel voor dit doel een flowchart nodig hebt. Met ieder jaar programmeer-ervaring kun je steeds beter zaken voor je gaan zien en in gedachten al oplossen.

Flowcharts voor communicatie

Maar de flowchart dient ook een ander doel; communicatie met je teamleden. Wanneer je samen aan één programma werkt is het belangrijk dat iedereen ook hetzelfde denkt over de oplossing en de werking van het programma. Eigenlijk dus: dat iedereen ‘hetzelfde voor zich ziet’.

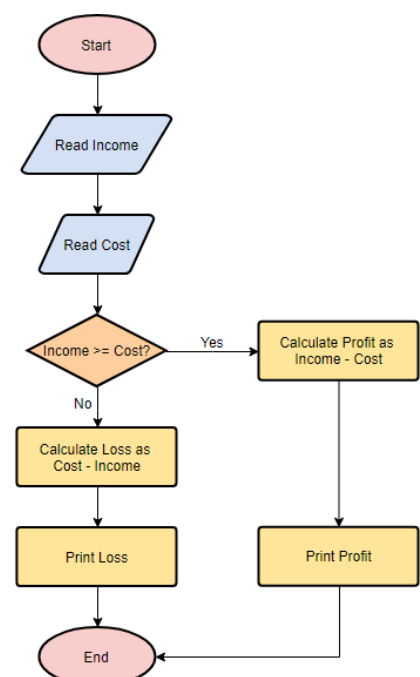
Een flowchart is een manier om jouw ideeën over het proces op papier te zetten. De rest van je team kan daar commentaar op geven en zo krijg je een goed gesprek over de werking van je programma.

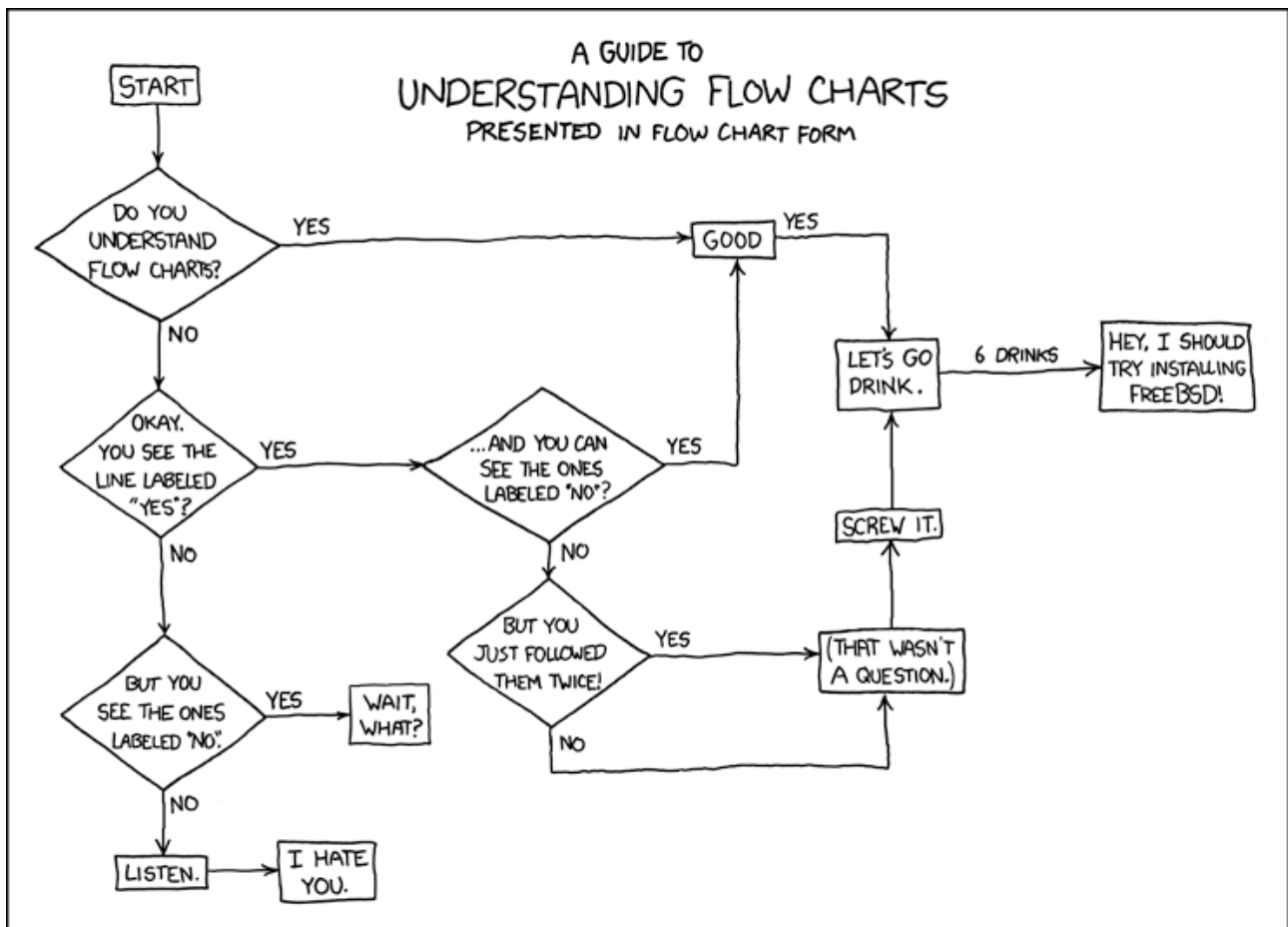
Uiteindelijk zorgen jullie ervoor dat je als team één flowchart hebt waar iedere programmeur zich aan zal houden bij het uitwerken van de code.

De ontwerpfase

De flowchart is één van de stappen die je in het V-model terugvindt bij de ontwerpfase. Je maakt een flowchart dus vaak vóórdat je aan het programmeren begint, maar het gebeurt ook weleens tijdens het werk.

Een flowchart is slechts een van de tools; je kent ook al het wireframe, dat is een tool om de “User Interface” van je applicatie te ontwerpen. In de volgende blokken leer je nog meer diagrammen, maar ze hebben allemaal hetzelfde doel: communicatie binnen het team én helpen om de complexiteit op papier te vangen.

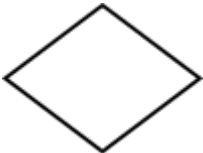




Bron: <https://xkcd.com/518/>

👉 2. De grammatica van een flowchart

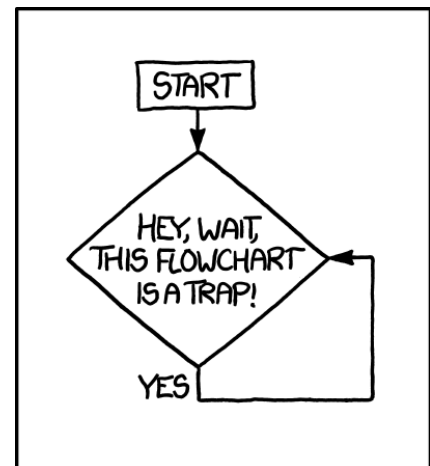
De bovenstaande uitleg van de stripmakers van *xkcd* geeft een 'eerste indruk' van hoe flowcharts werken. Voor de volledigheid zetten we hier de belangrijkste vier officiële symbolen op een rij:

Symbool	Naam (EN)	Naam (NL)	Uitleg
	<i>Terminal</i>	Begin/eind	Het begin en einde van een flowchart worden gevormd door een ovaal met daarin soms letterlijk het woord "Start" of "Einde". Vaak staat er andere tekst als "Druk op run" of "Programma sluit af".
	<i>Process</i>	Processtap	Eén van de stappen van het hele proces. Bijvoorbeeld: "tel 1 op bij getal", "bereken gemiddelde", "toon keuzemenu", enzovoort.
	<i>Decision</i>	Vraag	Meestal een if-statement; het programma kan hier kiezen tussen twee of meer routes. De vraag staat in de vorm, de mogelijke antwoorden staan op de uitgaande lijnen. Een <i>while</i> -loop herken je in flowcharts als vraag-symbool met een pijl die naar zichzelf terugverwijst.
	<i>Input/output</i>	Input/output	Een processtap waarbij gegevens worden ingelezen of aan de gebruiker gevraagd. Ook voor output: data wordt weggeschreven naar een bestand en/of het scherm.

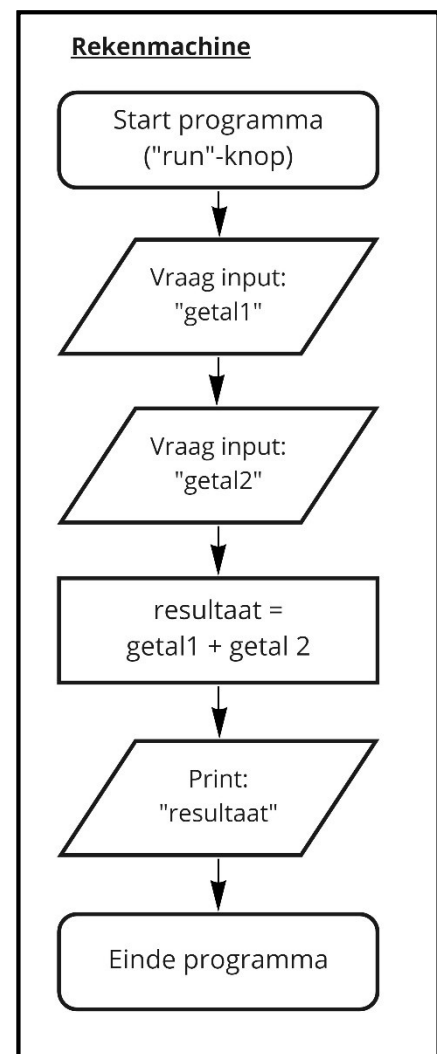
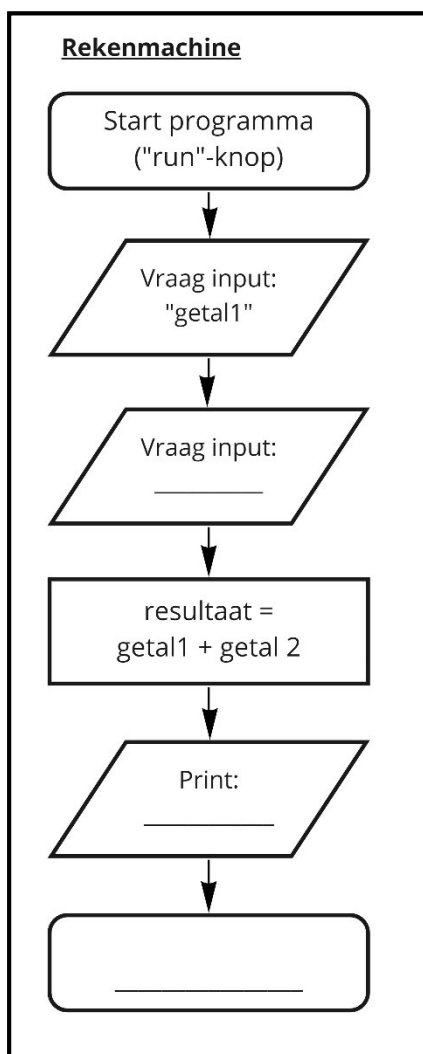
👉 3. Opdracht: flowcharts invullen

Op de volgende pagina's vind je een aantal flowcharts waarin nog stukken ingevuld moeten worden. Het zijn steeds flowcharts van programma's of opdrachten die je al kent.

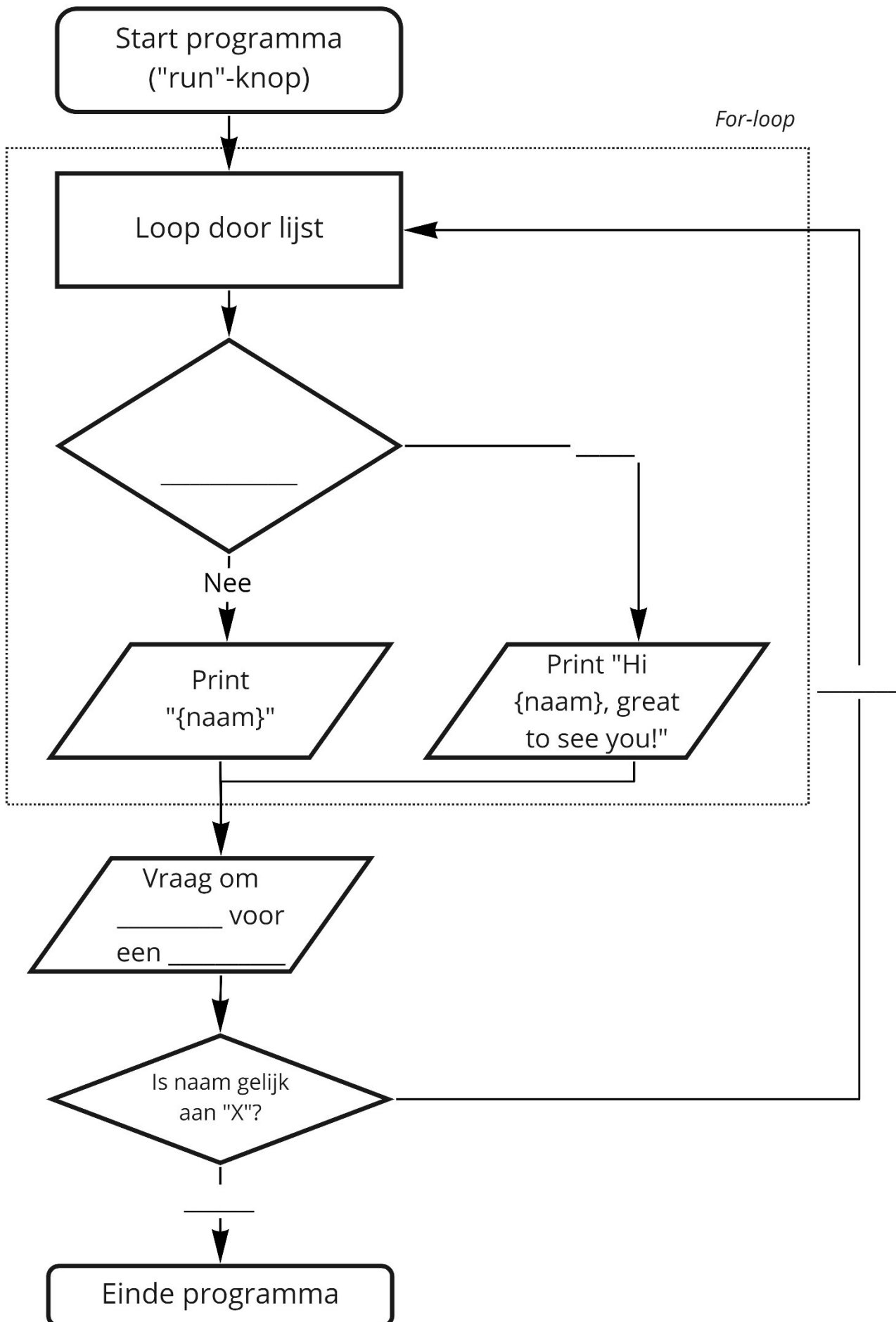
Maak de flowcharts kloppend door de blokken in te vullen. Soms moet je een heel blok tekenen, als een pijl nog nergens naartoe loopt. Je ziet hieronder een voorbeeld:



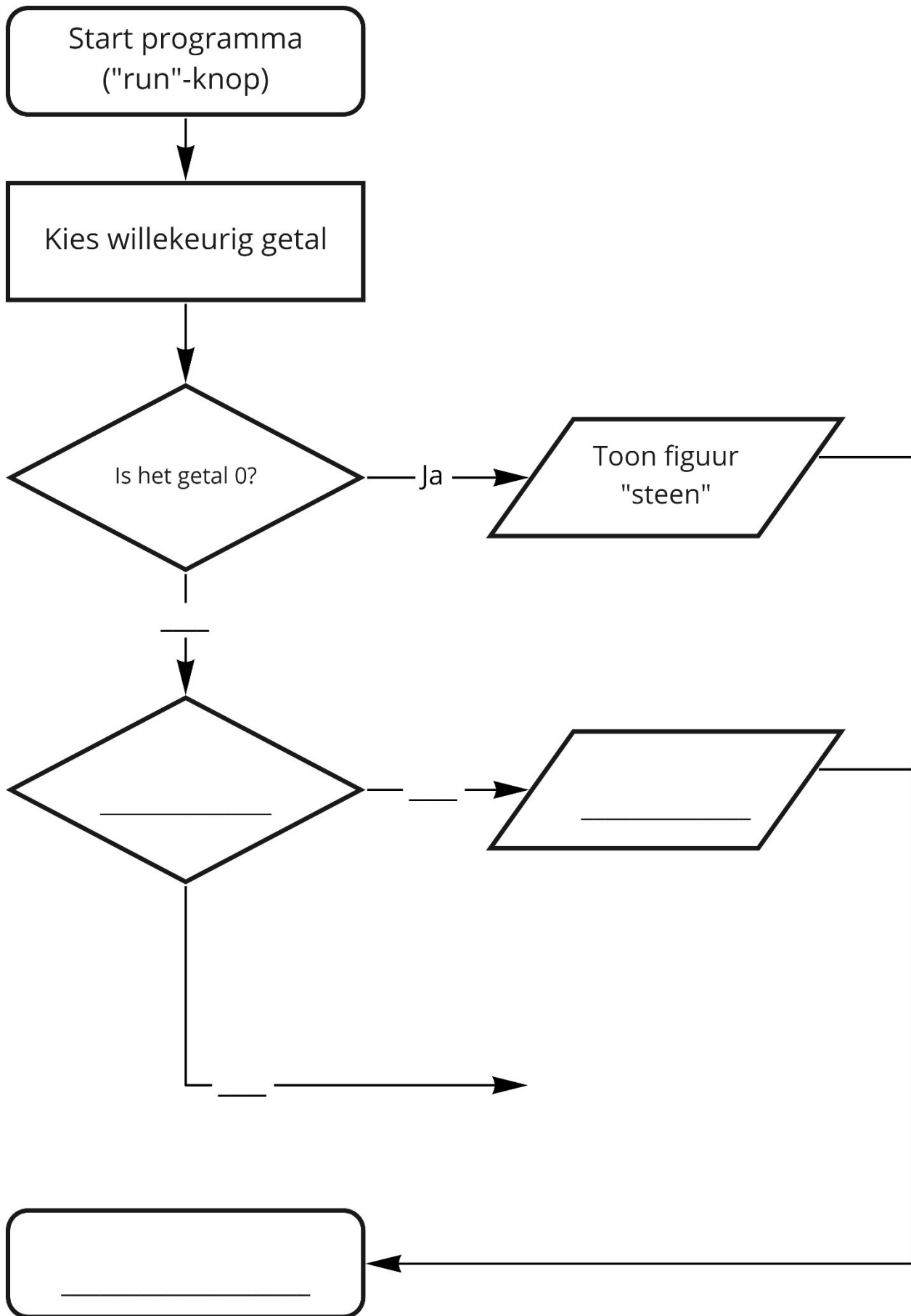
Bron: <https://xkcd.com/1195/>



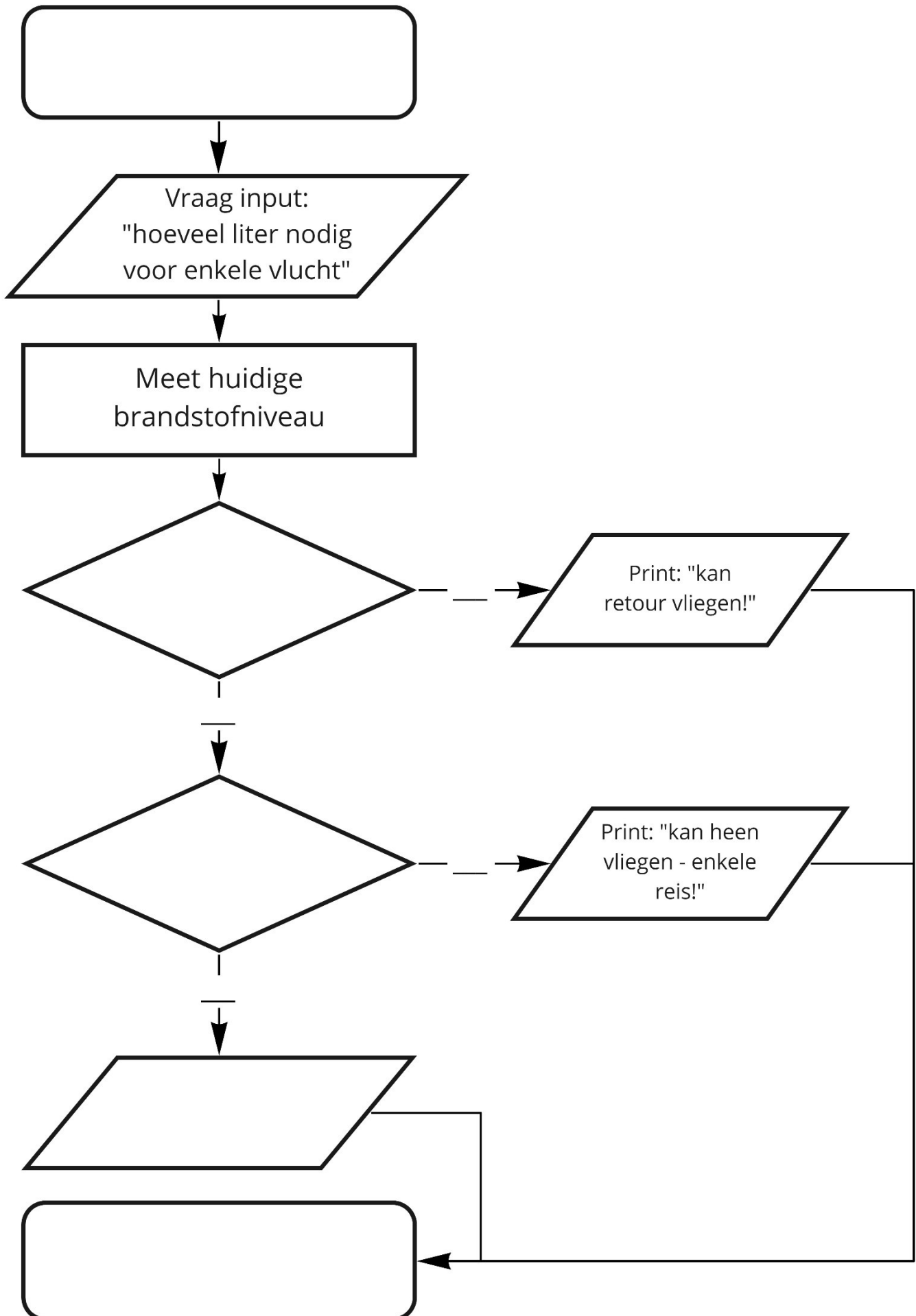
Pilotenlijst



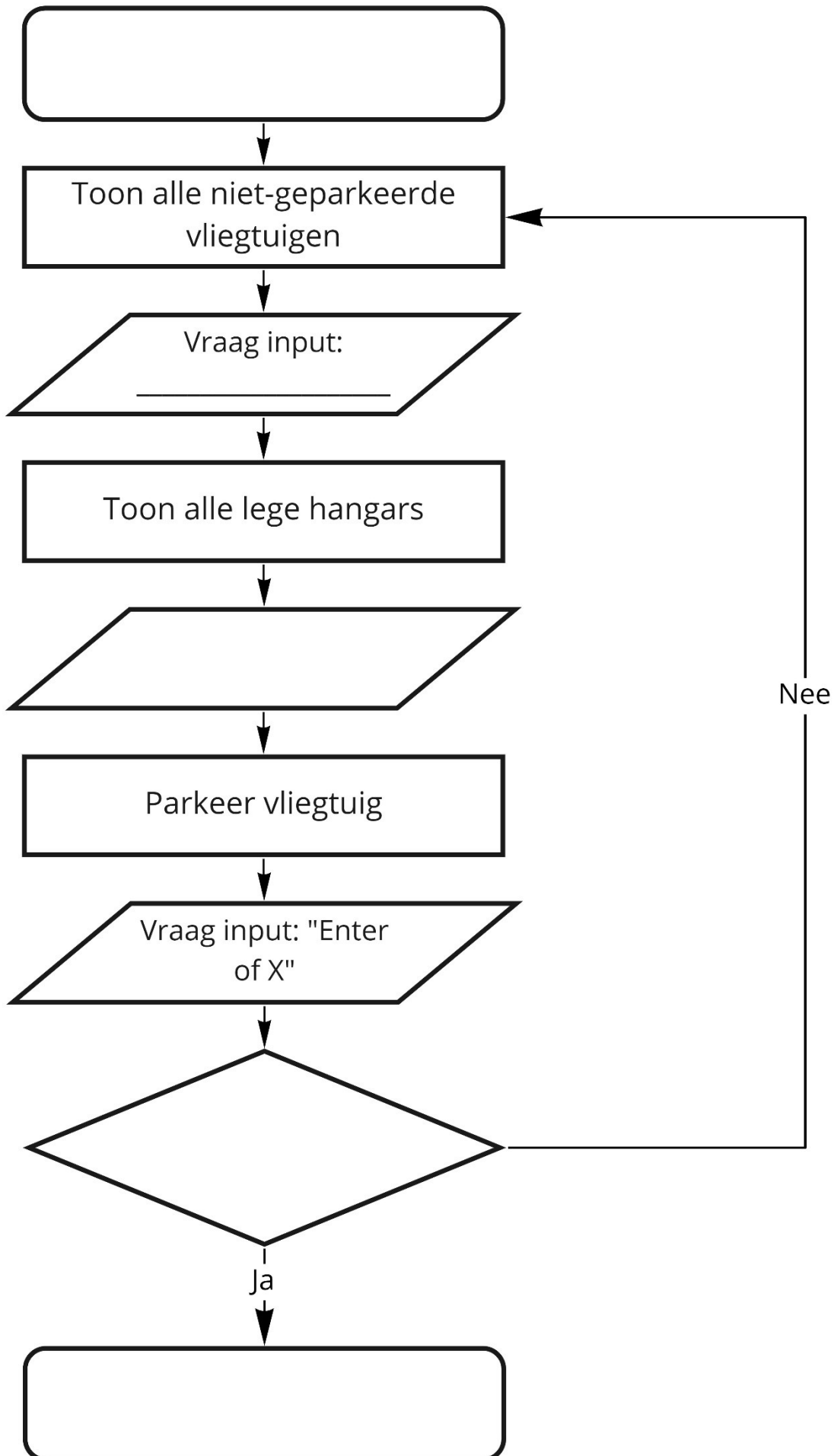
Steen-papier-schaar



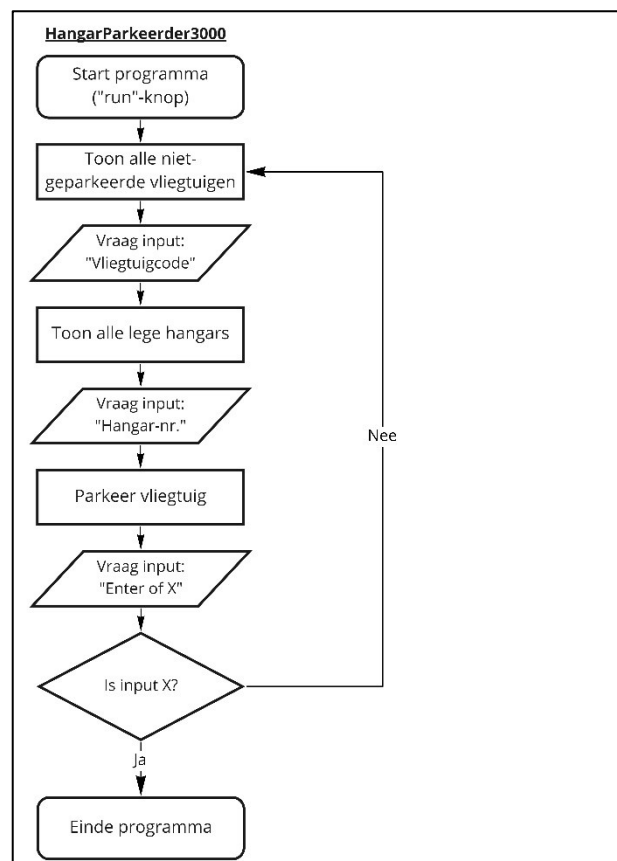
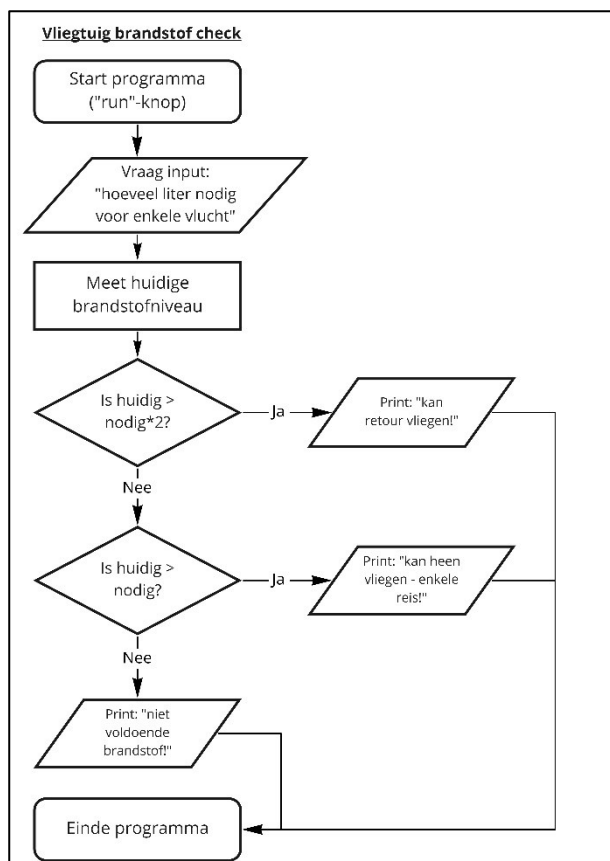
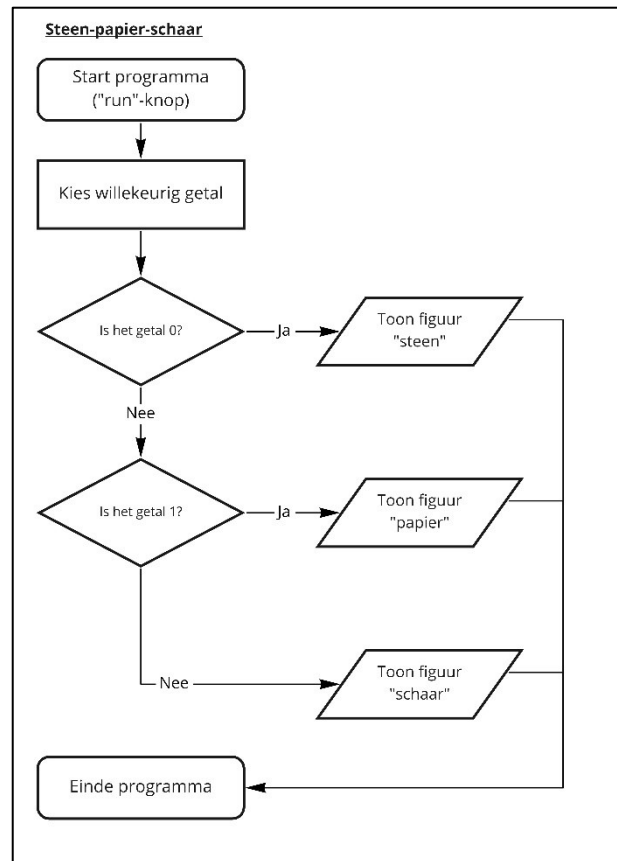
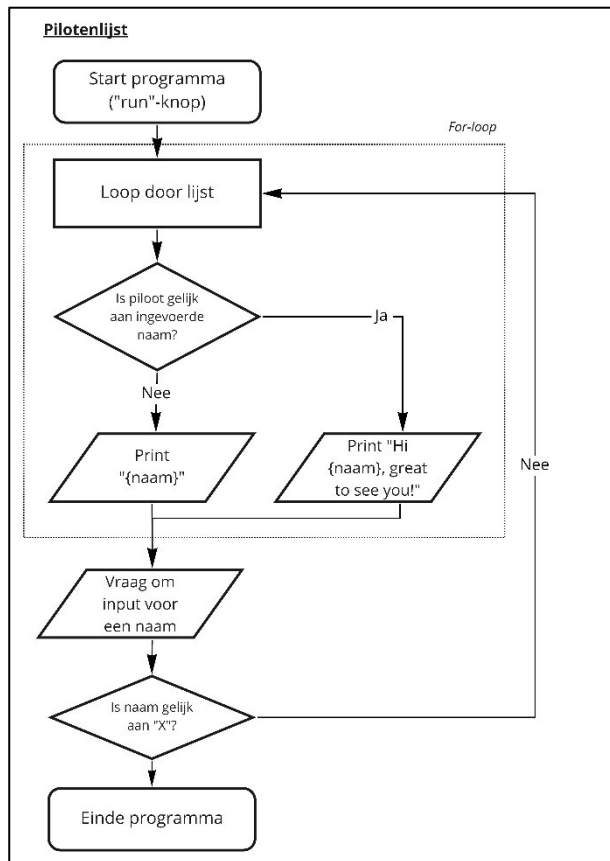
Vliegtuig brandstof check



HangarParkeerder3000



Oplossingen:



4. Flowchart op twee niveaus

In de laatste flowchart van opdracht 3 (de HangarParkeerder3000) staan een aantal processtappen die je niet met één regel code kunt uitwerken. Bijvoorbeeld: de stap “parkeer vliegtuig” bestaat eigenlijk uit een heleboel kleine stapjes:

Parkeer vliegtuig

Je ziet hier rechts dat het “parkeren” dus bestaat uit zes regels code. Je moet immers bij het vliegtuig het juiste hangar-nummer instellen en ook bij de hangar zelf opslaan dat deze nu ‘bezet’ is.

```
#Stel hangar-nr. in bij vliegtuig:
for plane in planesLanded:
    if plane['code'] == inputCode:
        plane['hangar'] = inputHangar

#Zet de hangar zelf op 'occupied':
for h in hangars:
    if h['num'] == inputHangar:
        h['occupied'] = True
```

Sub-programma

Omdat de processtap “Parkeer vliegtuig” dus weer uit een aantal stappen bestaat, kun je deze uitwerken in een eigen flowchart. Wanneer je een processtap verder uitwerkt heet dan een subroutine of sub-programma.

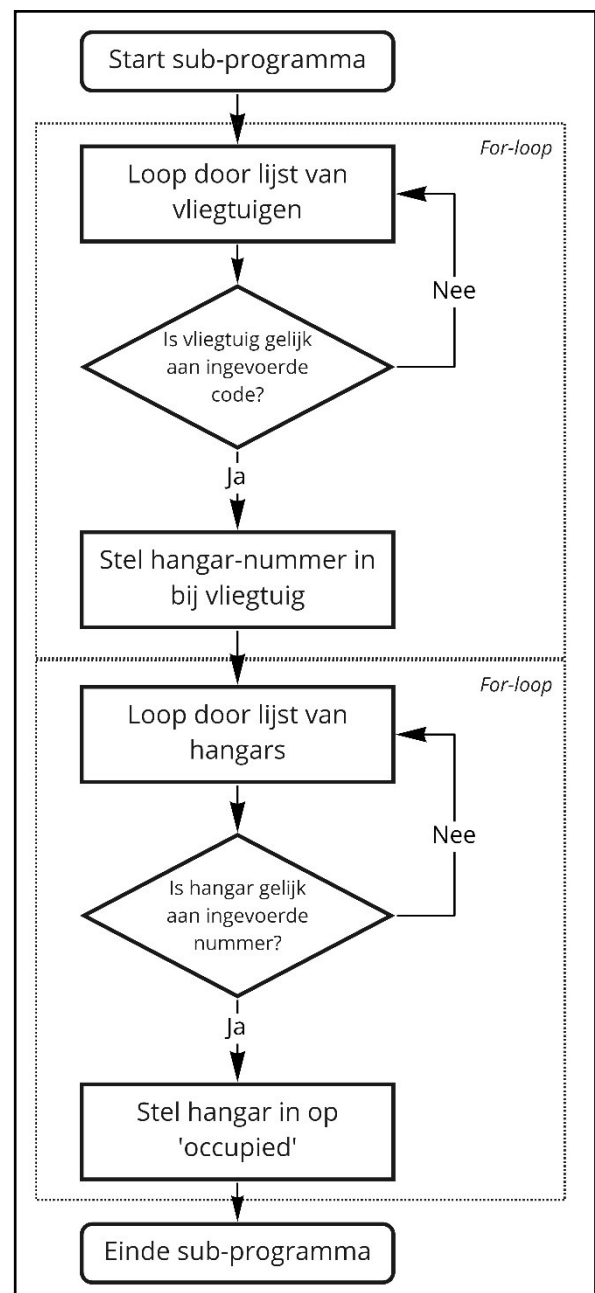
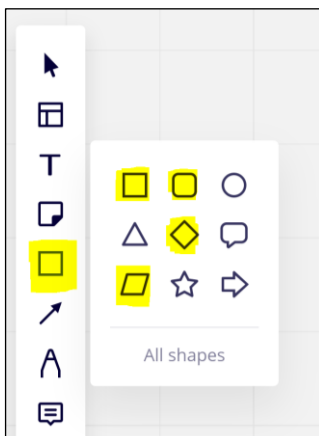
Twee niveaus

We hadden dus al een flowchart op “hoog” niveau, waarin je ziet wat de grote stappen van het programma zijn. Een aantal van die stappen kun je ook weer op “laag” niveau uitwerken, in meer detail dus.

Met deze werkwijze kun je makkelijk overzicht houden in je flowcharts. Want als je alle stappen tot in detail zou uitwerken in één flowchart, krijg je natuurlijk een heel groot diagram waar je geen wijs meer uit kunt.

5. Flowcharts digitaal tekenen

In de volgende opdracht ga je zelf een flowchart tekenen. We doen dat digitaal in het programma *Miro* (<https://miro.com/>). Je hebt al een account gemaakt in een vorige les. Is dat nog niet gelukt? Maak dan alsnog een account. Zodra je een nieuw bord hebt gestart, vind je aan de linkerkant de vier vormen die je nodig hebt:



6. Opdracht: van code naar flowchart

Je gaat van twee processtappen zelf een flowchart tekenen in *Miro*. Merk op dat deze stappen erg lijken op (een deel van) de stap die onder paragraaf 4 al is uitgetekend als voorbeeld.

1. Teken van deze processtap een flowchart. Je ziet ook uit welke regels code de stap bestaat:

Toon alle niet-geparkeerde vliegtuigen

```
print("Deze vliegtuigen wachten om geparkeerd te worden:")
for plane in planesland:
    if(plane['hangar'] == 0):
        print(f"{plane['code']} (type: {plane['type']})")
```

2. Teken van deze processtap een flowchart. Je ziet ook uit welke regels code de stap bestaat:

Toon alle lege hangars

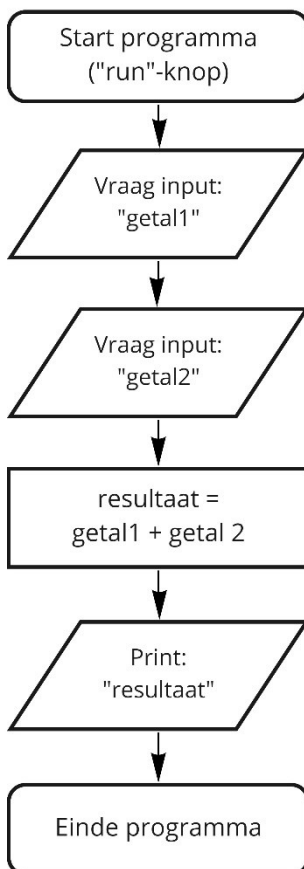
```
print("\nDeze hangars zijn nog vrij:")
for h in hangars:
    if h['occupied'] == False:
        print(f"Hangar {h['num']}")
```

7. Opdracht: van flowchart naar code

Hieronder zie je twee eenvoudige programma's als flowchart, werk deze uit in Python-code. Tip: in flowchart twee zie je vanaf de vraag een pijl terugverwijzen naar de input. Je kunt daarin een while-loop herkennen.

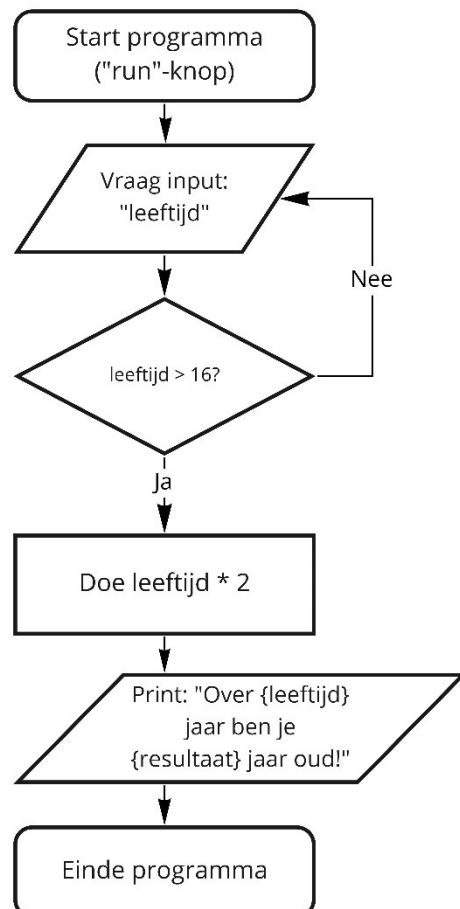
1:

Rekenmachine



2:

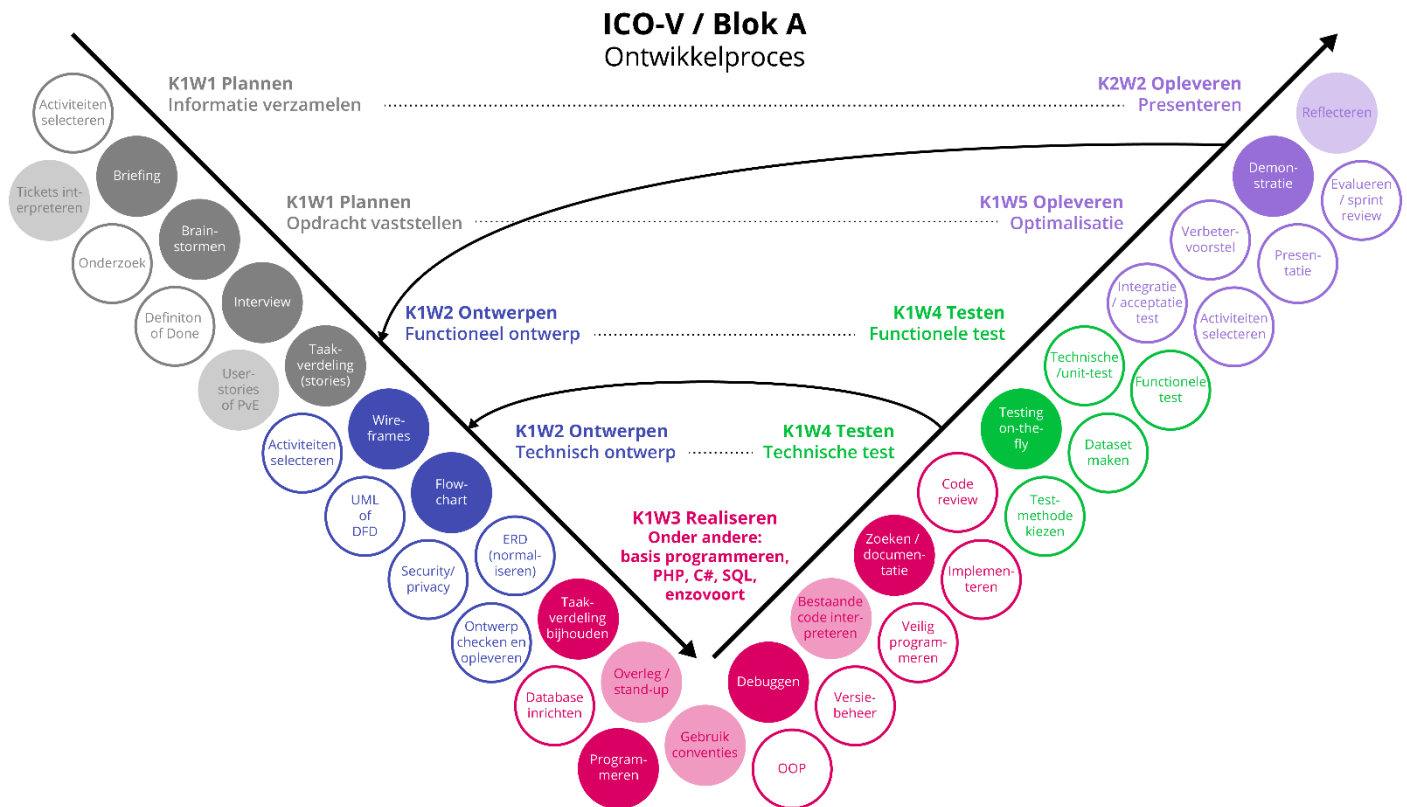
Leeftijd



4. Diversen en feedbackmoment

1. De ontwerpfase in het V-model

Het V-model beschrijft hoe we een ontwikkelproject aanpakken. Met een “ontwikkel”-project bedoelen we dan het ontwikkelen van software. Dat is méér dan alleen programmeren! In het V-model kun je dat ook goed zien; er zijn maar liefst vijf fases waarvan alleen de middelste gaat over programmeren (de realisatiefase).



Deze module heet ONT-I en gaat dus over de ontwerpfase. Je ziet dat er twee bolletjes actief zijn voor de ontwerpfase in blok A: wireframes en flowcharts. In latere blokken ga je nog veel meer technieken leren voor de ontwerpfase, die zul je vooral nodig hebben omdat we later ook veel complexere applicaties gaan maken.

De ontwerpfase valt in principe dus vóór de realisatiefase (het programmeren), maar je kunt ook zien dat er pijlen terugverwijzen naar de ontwerpfase. Soms gebeurt het dan ook dat je aan het werk bent met programmeren en toch iets wil wijzigen; je gaat dan heel even terug naar het ontwerpen om je wireframe of flowchart aan te passen.

Het kan ook voorkomen dat je helemaal geen ontwerp had gemaakt, maar er later achter komt dat je toch een ontwerp nodig hebt. Ook dan ga je terug naar de ontwerpfase om alsnog wat dingen op papier te zetten.

Flexibel

De ontwerpfase is dus een flexibele fase. Soms lever je vóór het programmeren begint al helemaal complete flowcharts op die netjes zijn uitgewerkt. Maar veel vaker zul je eerst wat schetsen maken, dan eens wat proberen te programmeren en dan weer terugkomen bij je ontwerp.

Het doel van de ontwerpfase

Het doel van de ontwerpfase is dan ook niet dat je na deze fase een hele mooie perfect uitgewerkte wireframe of flowchart hebt. De ontwerpfase draait vooral om communicatie binnen je team: je moet ervoor zorgen dat iedereen hetzelfde naar de opdracht kijkt en dat je het eens wordt over de aanpak.

Ook zorgt het vastleggen van je ontwerp voor minder denkwerk verderop in het project; als je de aanpak goed uitdenkt kun je daar tijdens het programmeren op terugvallen, in plaats van steeds opnieuw een oplossing te moeten bedenken.

👉 2. Wireframes: een opfrisser

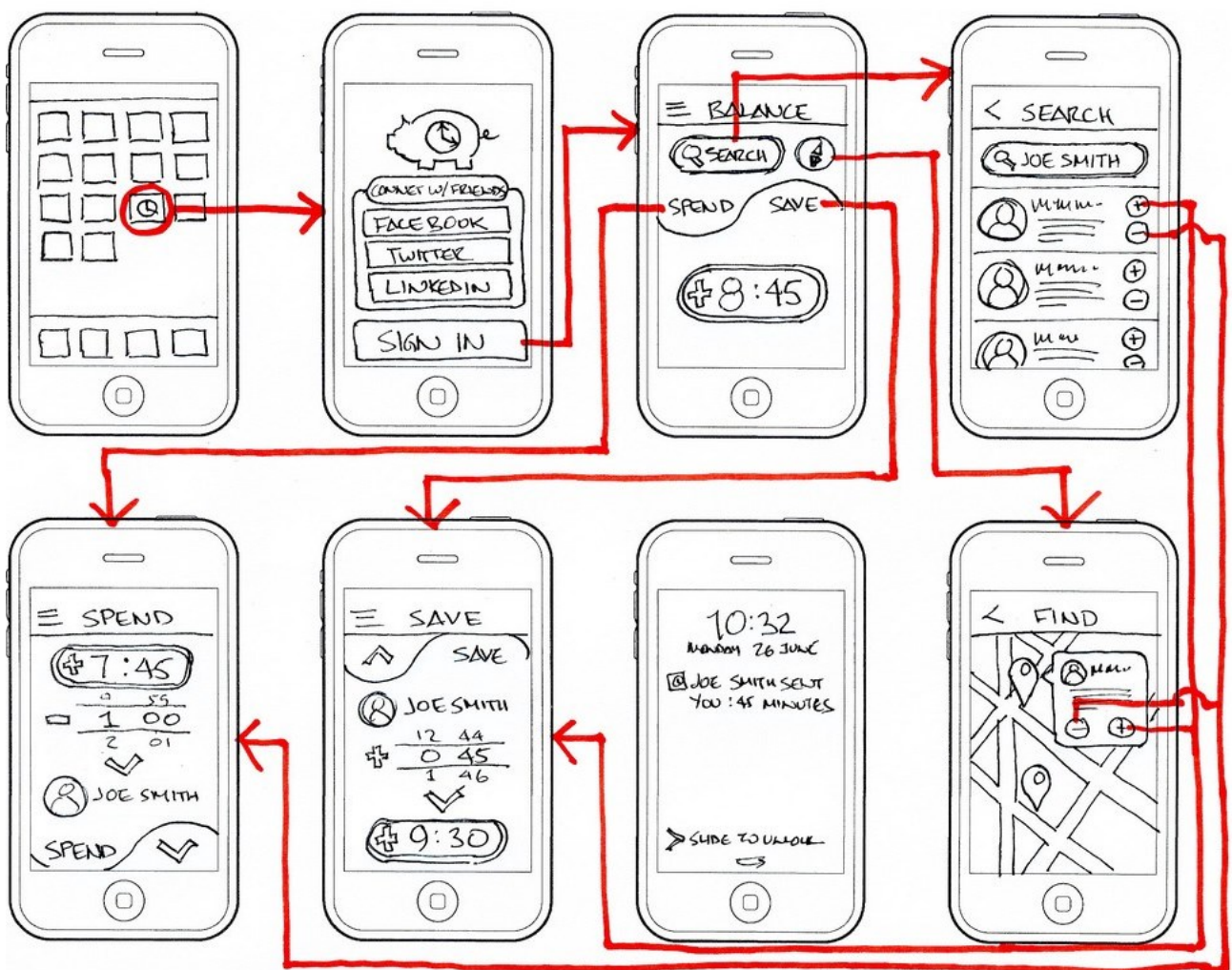
In hoofdstuk 1 zijn de wireframes behandeld, hier krijg je ook vragen over in het feedbackmoment. Kijk eventueel terug naar hoofdstuk 1 en lees zeker ook deze toevoeging:

Het niveau van detail

Voor de ontwerpfase geldt: als we spreken over een wireframe bedoelen we meestal een “low-fidelity” wireframe. Dat betekent dat je vaak toe kunt met een potloodschets van je webpagina waar nog niet zoveel details in zitten.

Maar: een schets moet wel *enig* niveau van detail bevatten. Dat wil zeggen dat een schets met twee of drie grote blokken niet akkoord is. Vooral bij menu's, formulieren en knoppen is het belangrijk om de lay-out toch te tekenen. Een wireframe moet namelijk laten zien hoe de applicatie of website gebruikt kan worden: op welke knoppen kun je drukken en wat gebeurt er dan?

Dat betekent dat je ook bijvoorbeeld een contactformulier niet af kunt doen door één vierkant te tekenen en daar “contactform” in te schrijven. Je ziet hieronder nog een goed voorbeeld, waarop niet véél details te vinden zijn, maar je kunt toch onderscheiden hoe de app werkt omdat alle knopjes enzovoort te vinden zijn:



👉 3. Opdracht: aandacht voor A3

Docent en klas staan waar nodig stil bij de praktijkopdracht A3. Moeten er nog zaken worden besproken of herhaald over de flowcharts?

👉 4. Opdracht: feedbackmoment

Maak het feedbackmoment. Meer informatie krijg je in de les van je docent.

5. Wireframes uitbreiden

1. Opdracht: nabespreken feedbackmoment

De docent bespreekt in de klas het feedbackmoment van vorige les en/of je krijgt inzicht in je antwoorden en behaalde punten. Noteer je leerpunten. Waar heb je punten laten liggen en hoe ga je daaraan werken:

2. Bestaand ontwerp aanpassen

Tot nu toe hebben we de stappen van de ontwerpfase steeds bekeken door de bril van een heel nieuw project, dat vanaf de grond opgebouwd wordt. Maar het komt in ons werkveld eigenlijk vaker voor dat je gaat werken aan een bestaand project. Dat kan bijvoorbeeld een website zijn die al helemaal af was, maar een tijdje later vraagt de klant je om een wijziging te doen. Dat kan variëren van een kleine wijziging (een foutje oplossen, een knop verplaatsen) tot veel grotere aanpassingen (een hele nieuw stijl doorvoeren, een nieuwe pagina toevoegen, een nieuwe functionaliteit bouwen).

Ook dan krijg je te maken met de plan- en ontwerpfases. Je moet immers goed uitzoeken wat de klant precies wil wijzigen (plannen) en hoe je dat gaat maken (ontwerpen). Vaak is er ook al een ontwerp voor de originele website, waar je rekening mee moet houden.

Documentatie aanpassen

Het is belangrijk dat je collega's (of bijvoorbeeld de stagiairs die na jou komen) ook weer verder kunnen werken aan een project. Daarom is het een goed gebruik om het ontwerp aan te passen als je een wijziging doet aan een bestaande applicatie of website. Zo blijft de ontwerp-documentatie altijd kloppend met het echte werk. Ook bij kleine wijzigingen is het goed om toch een update te doen.

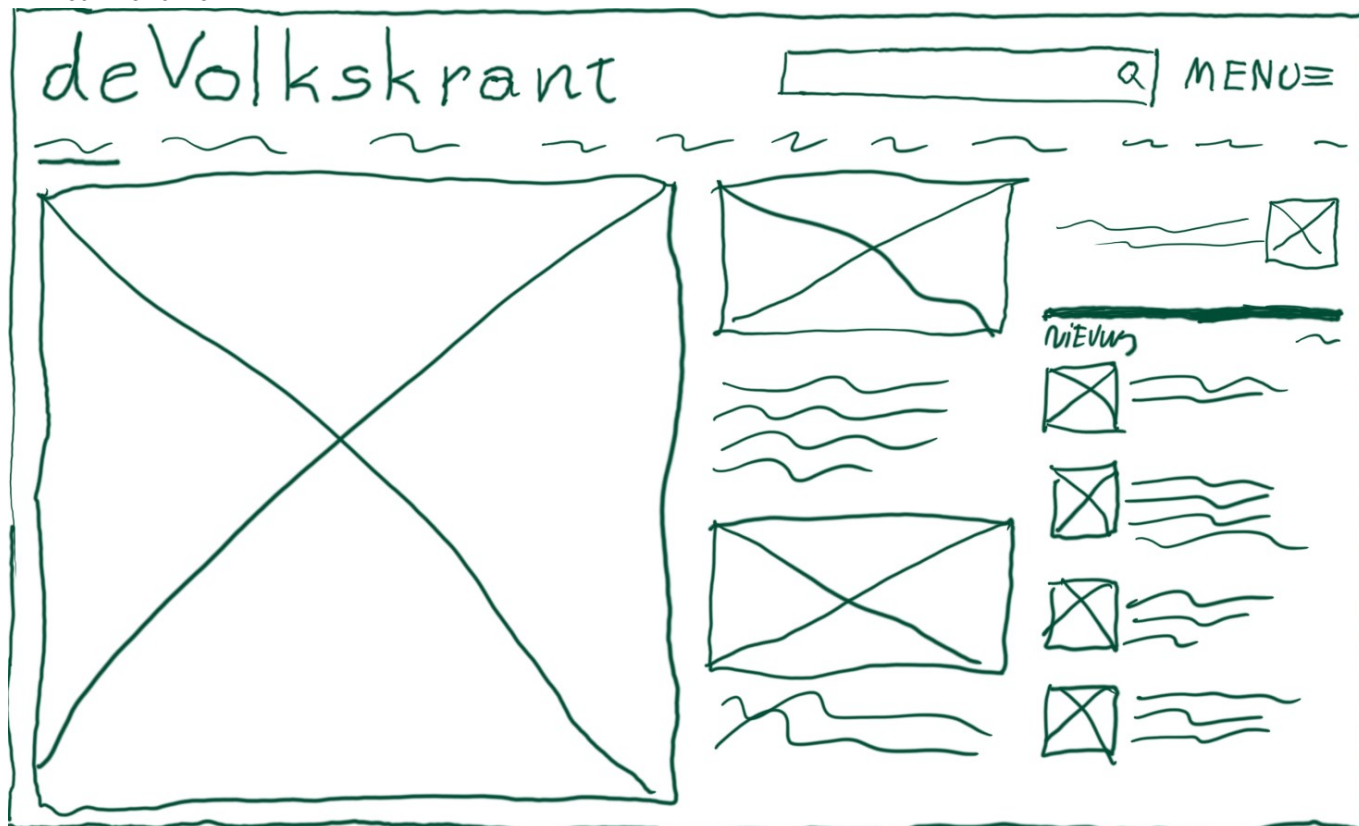
3. Opdracht: groepsgesprek

De docent bepaalt of jullie als klas of in duo's het volgende gaan bespreken. Stel je deze situatie voor en bespreek de vragen die daarna komen (dat betekent; niet enkel ja/nee antwoorden maar ook het 'waarom' onderzoeken terwijl je met elkaar in gesprek bent).

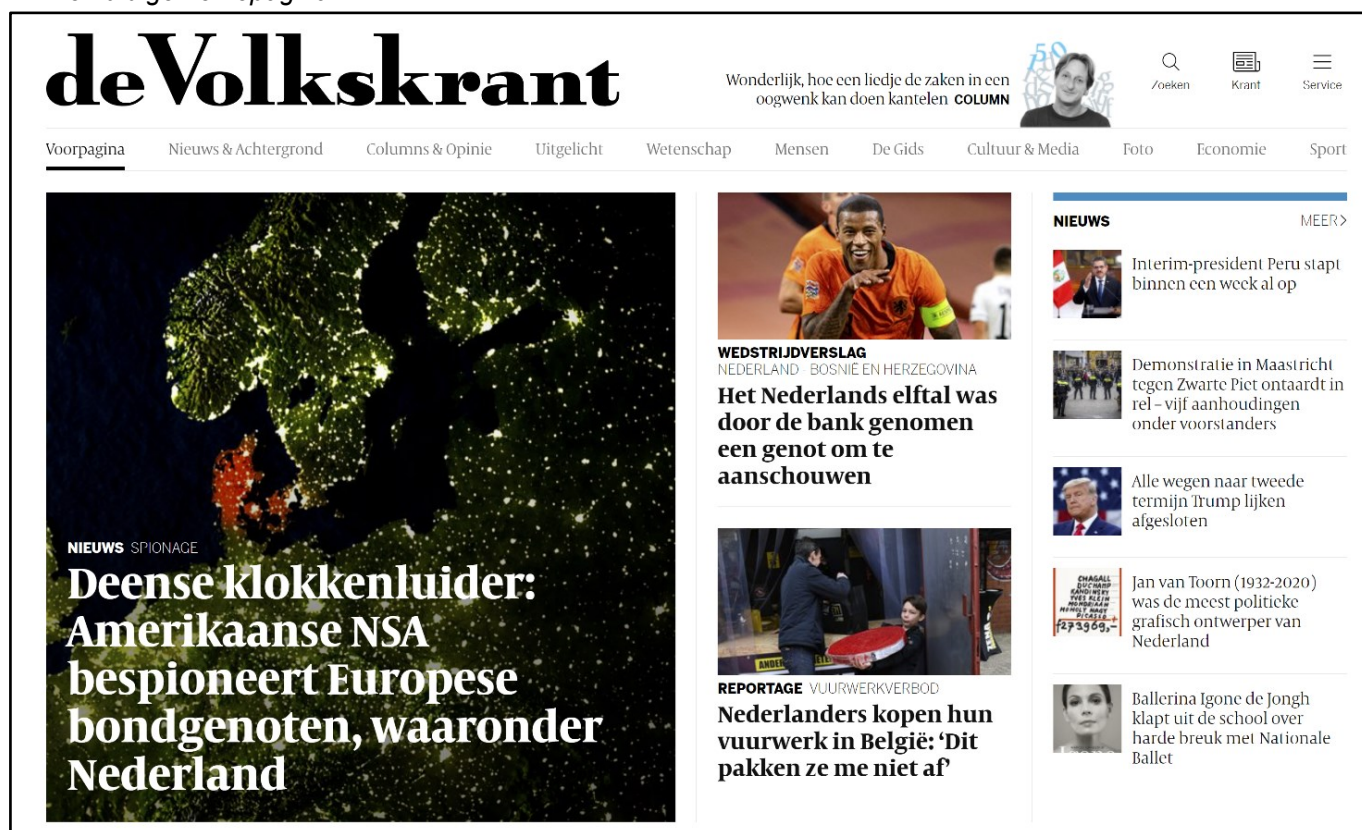
"Bij het bedrijf waar jullie werken komt de opdracht binnen om een aanpassing te doen aan de website van een Nederlandse krant. Men vraagt om het vak met icoontjes zo te maken: links moet het menu staan en rechts 'zoeken'. Dat zou logischer zijn voor de gebruikers. Bij de opdracht wordt ook de ontwerp-documentatie aangeleverd die de vorige developers hebben gemaakt. Op de volgende pagina vind je het originele wireframe en een screenshot van de homepage."

1. Is het aangeleverde wireframe nog actueel?
2. Zo nee, wat is er veranderd? Geef eventueel op de tekening aan.
3. Is het een probleem dat de pagina nu al anders is dan het wireframe?
4. In de opdracht wordt niet gesproken over het 'krant'-icoon, en het staat ook niet op de wireframe; wat zou daarmee moeten gebeuren? Moet het blijven staan? Is men vergeten het te benoemen omdat het niet op de wireframe staat, of is het bewust omdat het icoon weg moet?
5. Wat is het verstandigste; gewoon aan de slag gaan óf de klant contacteren om bevestiging te vragen over het 'krant'-icoon?
6. Kun je met deze opdracht en documentatie uit de voeten, of had je liever correcte documentatie gehad?

Het wireframe:



De huidige homepage:

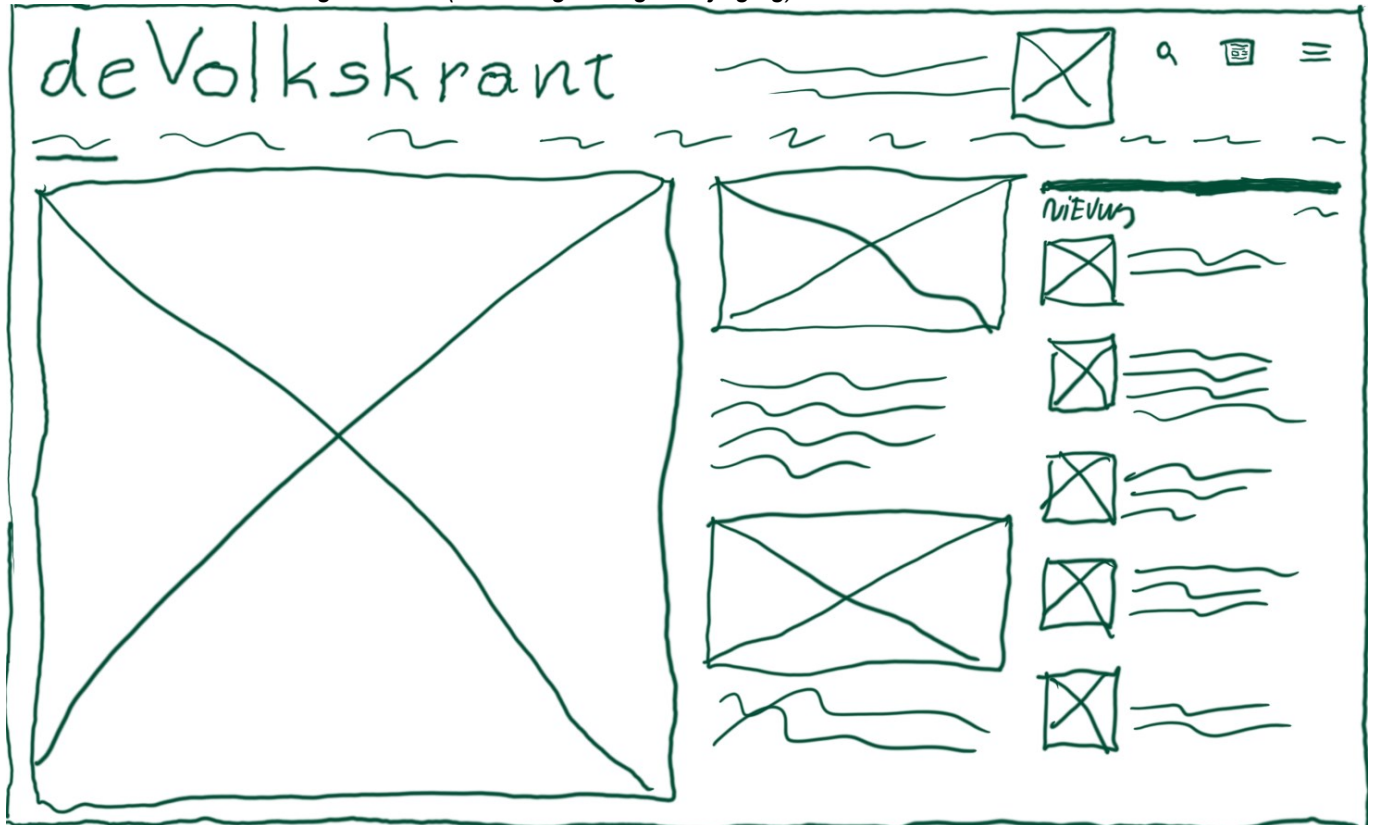


4. Opdracht: aanpassing doen aan wireframe

De docent bepaalt of jullie als klas of in duo's het volgende gaan bespreken. Stel je deze situatie voor en bespreek de vragen die daarna komen:

“Jullie hebben de vraag voorgelegd aan de opdrachtgever en een paar dagen later komt het antwoord; men verwacht dat op de huidige website het icoon voor zoeken en service worden omgewisseld. Bij de vorige developers heeft men gevraagd om nog een correct wireframe van de huidige website op te leveren, die vind je hieronder. Nu is het aan jullie om de wijziging te doen. De opdrachtgever benadrukt dat ze ná jullie wijziging natuurlijk weer een kloppende wireframe terugverwachten.”

Wireframe van de huidige situatie (vóór de gevraagde wijziging):



1. Hoe kun je nu het beste een aanpassing doen aan het wireframe?
2. Wat voor programma zou je kunnen gebruiken? Of werk je op papier?
3. Waar kun je tegenaan lopen bij het doen van een aanpassing en een bestaande tekening?

5. Opdracht: aanpassing doorvoeren

Voer daadwerkelijk de aanpassing in het wireframe door. Let op: we gaan de site zelf natuurlijk niet aanpassen, het gaat enkel om het bovenstaande wireframe waarin de icoontjes omgewisseld dienen te worden zodat het klopt met de gevraagde wijziging van de opdrachtgever.

6. Flowcharts revisited

1. Een flowchart maken 'from scratch'

Het maken van een flowchart kan lastig zijn als je helemaal 'from scratch' moet beginnen. Hoe ga je de flowchart dan opzetten, waar start je mee, enzovoort. Belangrijk daarbij is: start vooral. Blijf niet te lang nadenken over je aanpak, maar begin eens met een schets te maken op papier. Dat hoeft nog helemaal niet netjes te zijn, maar het kan je wel weer verder op weg helpen.

Twee doelen

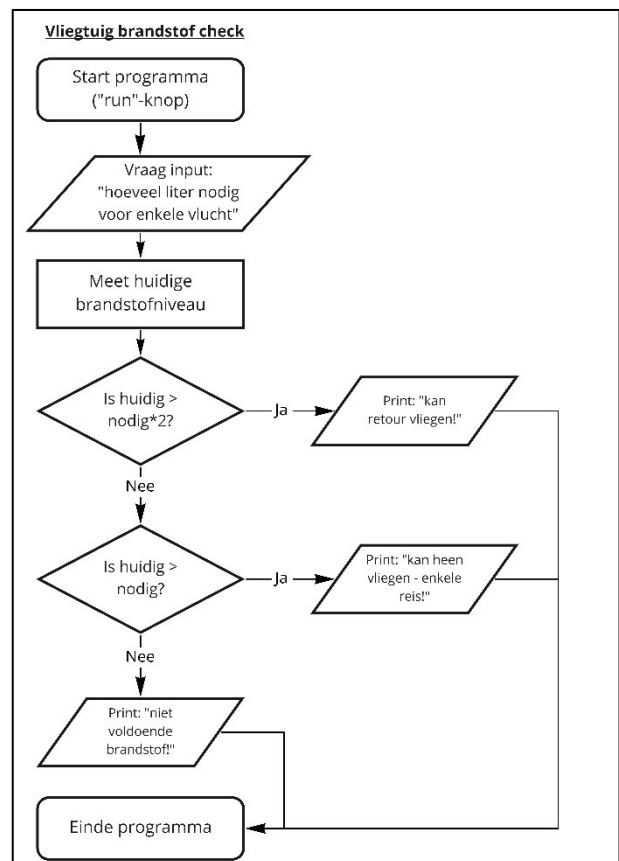
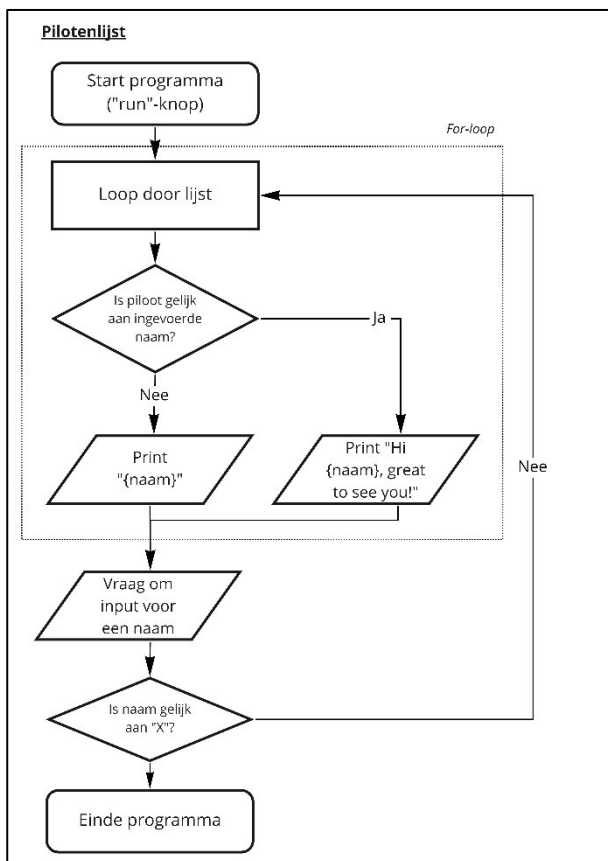
We weten dat een flowchart eigenlijk twee doelen dient: communicatie binnen je team én op papier 'vangen' van de complexiteit. Die twee dingen krijg je ook voor elkaar met een flowchart die vlug is geschetst op de spreekwoordelijke achterkant van een bierviltje. Je kunt dus gewoon beginnen met tekenen, vervolgens overleggen binnen je team en gedachten uitwisselen, en dan weer een nieuwe versie maken. Dat werkt beter dan wanneer je probeert in één keer een perfecte flowchart te maken. Dat is bovendien helemaal niet nodig, de flowchart (en andere ontwerpdocumenten) zijn *juist* bedoeld als onderwerp van gesprek.

Uiteindelijk uitwerken

We hebben óók geleerd (in hoofdstuk 4) dat het belangrijk is dat je ontwerpdocumentatie altijd up-to-date is. Zo kun je een project netjes overdragen aan eventuele opvolgers. Daarom moet je een flowchart uiteindelijk wel netjes digitaal uitwerken, dat is eigenlijk de *laatste* stap van het ontwerpproces.

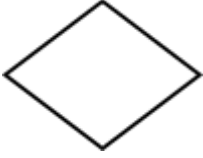
Voorbeelden

Om er weer even in te komen, kun je deze twee voorbeelden uit hoofdstuk 2 bekijken:



2. Opdracht: de grammatica van een flowchart

Hoe was het ook alweer? Vul de lege vakjes in de tabel in of aan:

Symbool	Naam (NL)	Uitleg
	_____	<i>Het begin en einde van een flowchart.</i>
	_____	<i>Eén van de stappen van het hele proces. Bijv.:</i> _____
	_____	_____
	_____	<i>Een processtap waarbij gegevens worden ingelezen of aan de gebruiker gevraagd. Ook voor output (naar scherm of bestand).</i>

3. Opdracht: flowchart maken voor PRA en vergelijken

We gaan in een aantal cycli werken richting een mooie flowchart voor de praktijkopdracht A5. Deze activiteit wordt door de docent aangestuurd qua stappen en tijdsindeling:

1. Lees jezelf in op de praktijkopdracht, zorg dat je ongeveer een beeld hebt van wat de bedoeling is.
2. Per persoon maak je een snelle schets van een flowchart. Het doel: de globale werking van het programma uittekenen, het hoeft nog niet 100% correct en ook nog niet heel netjes.
3. In de klas bekijken we een aantal schetsen. Wat valt op, wat zijn overeenkomsten en verschillen, wat klopt wel of niet?
4. Met je samenwerkingspartner voor PRA vergelijken jullie je schetsen. Maak hierna samen een iets uitgebreidere versie, neem wat meer tijd om ook netjes te werken maar het blijft een tekening.
5. In de klas bekijken we een aantal tekeningen. Wat valt op, wat zijn overeenkomsten en verschillen, wat klopt wel of niet?
6. Maak op papier eventueel aanpassingen met je team samen. Werk hierna digitaal een eerste versie uit. Deze kun je meenemen naar PRA om daar eventueel verder te werken.

4. Opdracht: check je schoolmail

Even iets wat eigenlijk niet bij deze module hoort, maar wel erg belangrijk is: check je schoolmail. Het einde van het blok nadert, dat betekent ook dat je per mail uitnodigingen kunt ontvangen voor bijvoorbeeld examen van een keuzedeel, gesprekken met je SLB, en andere belangrijke info. Stel eventueel in dat je mails worden doorgestuurd naar je persoonlijke mailadres.

III. Feedback-momenten

De feedback-momenten gaan in je portfolio zodat je voortgang duidelijk te zien is.

Feedback-moment FA27-PRO-12

Week: 12

Vorm: Theoretische toets

Vorbereiden: Kijk terug naar de theorie over wireframes (H1 / wk7), naast de theorie over flowcharts.

Let op: Het feedback-moment gaat over flowcharts én over wireframes.

In het feedback-moment zul je gevraagd worden om uit een aantal voorbeelden het beste wireframe te kiezen, op basis van de theorie. Ook krijg je meerkeuzevragen over de theorie. Voor flowcharts zul je voornamelijk invulvragen krijgen.

Leeruitkomsten

Aspect		Voldaan
12.02	Voor het functioneel ontwerp teken je <u>wireframes</u> waarin de lay-out van de applicatie te zien is, alsook de 'flow' tussen de schermen.	
Wireframes		De wireframes zijn 'lo-fi' getekend; vergelijkbaar met een potloodtekening. Goed te begrijpen, waar nodig voorzien van commentaar. Alle nodige schermen zijn uitgetekend.
Lay-out		Bevatten informatie over de besturing van de applicatie (knoppen en invoervelden). Dit is logisch en compleet.
Flow		De verbinding tussen verschillende schermen is duidelijk, de stappen die gebruiker neemt door de applicatie.
12.03	Voor het technisch ontwerp teken je een of meer <u>flowcharts</u> die de uitvoering van de code aangeven, inclusief te maken keuzes, herhalingen, enzovoort.	
Syntax		Je gebruikt de correcte syntax voor je flowchart.
Logica		De logica klopt en is begrijpelijk.
Oplossing		Het ontwerp biedt een oplossing voor de gestelde vraag / opdracht / probleem.

IV. Voor de docent

Spreiding over het blok

De lessen uit deze module worden verspreid over blok A gegeven. Dit betekent dat er op verschillende momenten in blok A verschillende hoofdstukken worden gegeven; de hoofdstukken zijn dus niet direct opeenvolgend.

Afdrukken

Dit boekje is bij uitstek bedoeld om op papier te gebruiken. Er zitten een aantal invuloefening in die digitaal moeilijk uit te voeren zijn.

V. Kwaliteit en verbeteracties

Versie	Aanpassingen voor opstart	Uitvoer	Punten na afloop
1.*	Nvt – eerste uitvoer	2020-nj	
2.*	lets duidelijker uitleg omtrent wireframes. Afschaffen moodboard, want geen onderdeel van V-model.	2021-vj	

curric